

Introduction to Matlab

Laboratory 1, DEDP

Table of contents

1 Objective	1
2 Theoretical aspects	2
2.1 The Matlab environment	2
2.2 Basic math: the Command Window as a calculator	2
2.3 Variables	3
2.4 Functions and keywords	4
2.5 Vectors and matrices	6
Creating vectors and matrices	6
The colon operator — regularly spaced sequences	7
linspace — a given number of points between two values	7
Sizes and dimensions	8
2.6 Operations on vectors and matrices	8
2.7 Indexing: accessing elements of a vector	9
Logical indexing	9
2.8 Basic graphics	10
2.9 Summary: functions and notation you should know	10
3 Exercises	12
4 Final questions	14

1 Objective

Get familiar with the Matlab environment: the Command Window, the Workspace and the Editor. Learn the basic Matlab syntax: arithmetic operators, variables, and the creation and manipulation of vectors and matrices.

Matlab is the tool we will use for the entire semester. Every laboratory builds on the basic operations learned here, so make sure that by the end of this session you can create a vector, operate on it element by element, and plot it — without looking at the notes.

Working style throughout the semester: type (do not copy-paste) every command yourself and read the output. if the result is not what you expected, that is a learning moment, not a bug.

2 Theoretical aspects

2.1 The Matlab environment

When you start Matlab, the desktop window contains three parts that we will use all semester:

- **Command Window** — the central pane where you type expressions and see the result immediately. Every command is typed after the prompt `>>`.
- **Workspace** — the side panel that lists all the variables that currently exist in memory, together with their size and value.
- **Editor** — a separate window (button “New Script” or `edit` in the Command Window) where you write multi-line collections of commands, called *scripts*. All labs are organized as scripts, and you are encouraged to work directly in the Editor and run the whole script (or a section) with the **Run** button or the F5 key.

Whenever you type an expression in the Command Window, Matlab *evaluates* it and prints the result. This is called **interactive mode**.

Some useful general commands:

- `clc` — clears the Command Window;
- `clear` — deletes all variables from the Workspace;
- `doc functionname` or `help functionname` — opens the documentation page of a function (e.g. `help sin`). There is no way to memorize all Matlab functions; you search through documentation to discover them.

2.2 Basic math: the Command Window as a calculator

Type the following expressions, one by one, and observe the results:

```
2 + 2

10 / 4

2^10

5 * (3 - 8)

sqrt(2)

sin(pi / 2)
```

Matlab has the usual arithmetic operators `+`, `-`, `*`, `/` and `^` (power), and the usual functions `sin`, `cos`, `tan`, `exp`, `log`, `sqrt`, `abs`, `round`, ... Parentheses `()` are used for grouping, exactly as in mathematics.

Unlike a common calculator, Matlab knows the special constants `pi`, `Inf` (infinity) and `NaN` (Not a Number, e.g. the result of `0/0`). Try it:

```
pi

1/0

0/0
```

Note: Matlab convention uses `log(x)` for the natural logarithm ($\ln x$). The base-10 logarithm is `log10(x)`.

2.3 Variables

A variable is a name (starting with a letter) that stores a value. Assign a value with the `=` operator:

```
a = 3

b = 2*a^2 + 5

name = "DEDP"
```

The value of the variable `a` can be used in any later expression. Notice that after each assignment the result is printed in the Command Window, and the new variable appears in the Workspace panel.

If you put a semicolon ; at the end of an assignment, Matlab still stores the variable, but **does not print** the result:

```
a = 3;  
b = 2*a^2 + 5;
```

This is the standard style inside scripts: variables are created silently, and we print only what we want to see.

Inspect your variables in the Workspace panel, or with the commands **who** (list names only) and **whos** (names, sizes, and memory):

```
whos
```

2.4 Functions and keywords

A **function** is a named operation that takes input values (arguments, in parentheses) and returns an output. Matlab has hundreds of built-in functions for any mathematical operation: we already met **sin**, **cos**, **sqrt**, **exp**, **log**, **abs**, **round**, **size**, **length**, **numel**, ...

A function can have several arguments and several outputs. The outputs are wrapped in square brackets:

```
% One argument, one output  
x = sin(pi/2)  
  
% Two arguments, one output  
m = mod(17, 5)      % remainder of 17 divided by 5: 2  
  
% One argument, two outputs: the maximum value and its position  
v = [3, 9, 1, 7];  
[mx, idx] = max(v)   % mx = 9, idx = 2
```

Try **help mod** and **help max** to see how the built-in help page looks. The page always shows the calling syntax of the function, i.e. how many arguments it accepts and in what order.

A **keyword** is a word that has a fixed, built-in meaning in the Matlab language and cannot be used as a variable name. The most common keywords, which will be used constantly in the following labs, are:

- **if**, **else**, **elseif**, **end** — conditional execution and the end of a block;
- **for**, **while** — loops (repeated execution);
- **function** — the definition of a user function;

- `return`, `break`, `continue` — control flow inside functions and loops;
- `true`, `false` — the logical values.

For example, the conditional structure has the form:

```
x = 3;
if x > 0
    disp('x is positive')
else
    disp('x is not positive')
end
```

Loops repeat a block of commands. A `for` loop repeats a fixed number of times, iterating over the elements of a vector:

```
% Sum the numbers from 1 to 10
s = 0;
for k = 1:10
    s = s + k;
end
disp(s)      % 55
```

A `while` loop repeats as long as a condition is true (the condition is re-evaluated before every repetition):

```
% First power of 2 that exceeds 100
p = 1;
while p <= 100
    p = 2*p;
end
disp(p)      % 128
```

Notice that `for`, `while`, `if` and `function` all delimit their block with the keyword `end` (or `end,else`, alternately for `if`): this is a general rule of the Matlab language.

A **user-defined function** is a function that we write ourselves, with the keyword `function`. A user function is normally written in its own file, with the same name as the function (e.g. a function called `myfunc` is stored in the file `myfunc.m`). As an example, the following file `mymean.m` defines a function that computes the mean of the elements of a vector:

```
% File: mymean.m

function m = mymean(x)
% MYMEAN Mean of the elements of the vector x
    m = sum(x) / length(x);
end
```

Once the file is saved, the function is called from the Command Window or from any script, exactly like a built-in function:

```
v = [1, 2, 3, 4, 5];
m = mymean(v)          % 3
```

The first line of a function is called the **signature**: it declares the output arguments (here `m`), the function name (`mymean`), and the input arguments (here `x`). The comment line below it is the so-called H1 help line, which is what `help mymean` displays in the Command Window. The inputs `x` and the output `m` are variables local to the function: they do not appear in the Workspace, and do not affect the rest of the script.

Note: the `disp()` function prints a value or a text in the Command Window, and `%` starts a comment (everything after `%` on a line is ignored by Matlab). You will see both in every script from now on.

2.5 Vectors and matrices

Vectors are the most important Matlab data type for this course: **a signal is a vector**, and **an image is a matrix**. Every laboratory will manipulate vectors, so this section is the foundation of everything that follows.

Creating vectors and matrices

A **row vector** is created with square brackets `[]`, the elements separated by spaces or commas:

```
v = [1, 2, 3, 4, 5]
w = [1 2 3 4 5]
```

A **column vector** separates the elements with semicolons:

```
v = [1; 2; 3; 4; 5]
```

A **matrix** combines the two: rows are separated by semicolons, and the elements of a row by spaces or commas:

```
M = [1, 2, 3; 4, 5, 6; 7, 8, 9]
```

Question: what is the difference between the row vector [1, 2, 3] and the column vector [1; 2; 3]?

The colon operator — regularly spaced sequences

The colon operator `a:step:b` creates a vector with the values $a, a + \text{step}, a + 2 \cdot \text{step}, \dots$ up to `b`. The middle value is the **step**:

```
x = 0:1:10
x = 0:0.1:1
x = 10:-2:0
```

Note that a negative step generates a decreasing sequence. If the step is omitted, it is 1: `0:10` is the same as `0:1:10`. The last value of the vector never exceeds `b` (for a positive step), so the number of elements is not necessarily fixed.

The colon operator is the standard way to build a **time axis** for generating signals:

```
t = 0:0.01:1;      % one second of time, sampled every 0.01s (100 samples)
x = sin(2*pi*5*t); % a 5 Hz sine wave, sampled on the axis t
plot(t, x)
```

linspace — a given number of points between two values

The function `linspace(a, b, n)` creates a vector with exactly n regularly spaced values between `a` and `b`, **including both endpoints**:

```
x = linspace(0, 1, 5)      % 5 points: 0, 0.25, 0.5, 0.75, 1
x = linspace(0, 1, 100)    % 100 points, step 1/99
```

The difference from the colon operator:

- `a:step:b` — we fix the **step** and let the number of points follow;
- `linspace(a, b, n)` — we fix the **number of points** and let the step follow (the step is always $(b - a)/(n - 1)$).
- `linspace(a, b, n)` always contains the endpoints `a` and `b`, while the colon operator may or may not reach exactly `b`.

Both produce the same kind of regular sequence, and both are used for generating time axes and sampling signals, so it is good practice to be comfortable with both.

Sizes and dimensions

The size of an array is obtained with `size()`, and the total number of elements with `numel()`:

```
M = [1, 2, 3; 4, 5, 6; 7, 8, 9];
size(M)

numel(M)

length(M)    % size along the largest dimension
```

A scalar is a 1×1 matrix: `size(5)` returns `1 1`.

2.6 Operations on vectors and matrices

Matlab treats every variable as a matrix. This gives two completely different types of operations, and *confusing them is the most common beginner mistake*:

- **matrix (linear algebra) operations**: `*`, `/`, `^` — the usual matrix product, inverse, power;
- **element-wise operations**: `.*`, `./`, `.^` — the same operation applied separately to each element (notice the dot!);

For scalars (single numbers) the two are identical, which is why we could write `2^10` above. For vectors and matrices they are different:

```
a = [1, 2, 3];
b = [4, 5, 6];

a .* b          % element-wise product: [4, 10, 18]

a .^ 2          % square each element: [1, 4, 9]
```

```
a * b           % ERROR: matrix product of 1x3 with 1x3 is undefined
```

The element-wise forms are the ones used for signals: they apply an operation to every sample of a signal.

2.7 Indexing: accessing elements of a vector

The elements of a vector are accessed with parentheses (). Matlab indexes from **1**:

```
v = [10, 20, 30, 40, 50];  
v(1)      % the first element: 10  
  
v(3)      % the third element: 30  
  
v(2:4)    % elements 2, 3, 4  
  
v(end)    % the last element
```

For a matrix we use two indices, row and column:

```
M = [1, 2, 3; 4, 5, 6];  
M(1, 2)    % row 1, column 2: 2  
  
M(2, :)    % everything on row 2: 4 5 6  
  
M(:, 3)    % everything on column 3: 3 6
```

The colon : alone means “all elements along that dimension”.

Logical indexing

A vector can also be indexed with a *logical* vector (a vector of **true/false** values), which selects only the elements where the logical value is **true**. This gives a very concise way to filter data:

```
v = [10, 3, 25, 8, 30, 1];  
sel = v > 8;          % logical vector: which elements exceed 8?  
v(sel)                % the elements themselves: 10 25 30  
  
v(v > 8)              % same thing, written in one line
```

Logical indexing is the tool we will use later for finding outliers, counting false alarms, extracting image regions, etc.

2.8 Basic graphics

The function `plot()` draws a vector as a sequence of samples, connected by a line:

```
t = 0:0.01:1;
x = sin(2*pi*3*t);
plot(t, x)
```

When you run this, a figure window opens. A few things to remember:

- the x axis of the plot **is not automatic**: it is given by the first argument (here the time axis `t`). `plot(x)` without a time axis would just plot the samples against their index;
- the figure can be annotated with `xlabel`, `ylabel`, `title`, and `grid on`;
- `stem(t, x)` is another useful plot for discrete signals, which draws a marker for each sample instead of a line. Use `stem` when you want to see the individual *samples* of a signal, and `plot` when you want to see the *shape*.

```
t = 0:0.01:1;
x = sin(2*pi*3*t);
plot(t, x)
xlabel('Time (s)')
ylabel('Amplitude')
title('A 3 Hz sine wave')
grid on
```

2.9 Summary: functions and notation you should know

The table below gathers everything introduced in this laboratory. You will use these functions and this notation in every other laboratory of this course, so keep it as a reference.

Signature	Call example	Description
Operators		
<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>^</code>	<code>a + b</code> , <code>a * b</code> , <code>a ^ 2</code>	Basic arithmetic operators; the Command Window works like a calculator

Signature	Call example	Description
<code>.* ./ .^</code>	<code>u .* v, x .^ 2</code>	Element-wise operations: applied to each pair of elements of equal-size arrays (matrix operators without the dot work differently)
Environment		
<code>clc</code>	<code>clc</code>	Clears the text in the Command Window
<code>clear</code>	<code>clear, clear x y</code>	Deletes all (or some) variables from the Workspace
<code>help</code>	<code>help plot</code>	Displays documentation for a function
<code>who, whos</code>	<code>whos</code>	Lists the current variables: who names only, whos adds size and class
<code>disp</code>	<code>disp(x)</code>	Prints the value of an expression or variable
Basics		
<code>abs, sqrt</code>	<code>abs(-3), sqrt(2)</code>	Absolute value, square root
<code>sin, cos, exp, log</code>	<code>sin(pi/2), log(10)</code>	Trigonometric and exponential functions; log is the natural logarithm (base <i>e</i>), base-10 needs log10
<code>round</code>	<code>round(2.6)</code>	Rounds to the nearest integer
<code>pi, Inf, NaN</code>	<code>2*pi, 1/0, 0/0</code>	Predefined constants
<code>size, numel, length</code>	<code>size(M), numel(M), length(v)</code>	Dimensions of an array: size (n by m), number of elements, length of the largest dimension
<code>max</code>	<code>[mx, idx] = max(v)</code>	Maximum value, and (with a second output) its position
Creating and indexing arrays		
<code>:</code> (colon)	<code>1:10, 0:0.1:1, 5:-1:1</code>	Creates a regularly spaced sequence a:step:b ; the middle value is the step, the last value never exceeds b
<code>[...]</code>	<code>[1 2 3], [1;2;3], [1 2; 3 4]</code>	Creates a row vector (spaces or commas), a column vector (;), or a matrix (; separates rows)

Signature	Call example	Description
<code>linspace(a,b,n)</code>	<code>linspace(0,1,5)</code>	Creates exactly n points evenly spaced between a and b , including both endpoints
<code>v(i), v(i:j)</code>	<code>v(3), v(2:4)</code>	Indexing: element i , slice from i to j
<code>end</code>	<code>v(end), M(end,:)</code>	Refers to the last position of an array
<code>M(i,:), M(:,j)</code>	<code>M(3,:), M(:,2)</code>	Whole row i / whole column j of a matrix
logical indexing	<code>v(v > 8), sel = v > 8; v(sel)</code>	Selects the elements for which a condition is true (true/false are 1/0 in arithmetic)
Graphics		
<code>plot(t,x)</code>	<code>plot(t, x)</code>	Plots samples connected by a line; the first argument is the axis
<code>stem(t,x)</code>	<code>stem(t, x)</code>	Plots each sample as a discrete marker; use for discrete signals
<code>xlabel, ylabel, title, grid</code>	<code>xlabel('Time (s)'), title('...'), grid on</code>	Annotations and grid for figures

3 Exercises

Work in the Editor: write a script (or a live script `.mlx`) that contains all the commands below, run it, and observe the results. Keep the script — it is the starting point of your individual work in this course.

1. Compute, in the Command Window:

- $7 \cdot 8 + 12$
- $(2^5 - 3^4) \cdot 6$
- $\frac{1}{1^2} + \frac{1}{2^2} + \frac{1}{3^2} + \dots + \frac{1}{10^2}$ (tip: compute it with vectors and `sum()`, or with a `for` loop)
- $\frac{4}{\pi^2}$ and compare it with the result at point c. What do you recognize?

2. Vectors and matrices:

- a. Create the row vector `a = [3, 1, 4, 1, 5]` and the column vector `b = [9; 2; 6; 5; 3]`.
- b. Create the matrix

$$M = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$$

- c. Using `size()`, display the sizes of `a`, `b` and `M`.
 - d. Extract from `M` the last column, the first two rows, and the element on row 2, column 3.
3. Create the vector `x = 0:1:10` (11 elements, from 0 to 10).
 - a. Display its length with `length()`.
 - b. Compute `sin(x*pi/4)` and round the values to integers with `round()`. What do you notice about the rounded values?
 - c. Compute, in one command, the sum of the elements of `x`, using the function `sum()`. (The function `sum()` adds all the elements of a vector — try to guess its meaning as a name for “the total”.)
 4. Signals as vectors:
 - a. Create a time axis `t = 0:0.001:1` (1001 samples).
 - b. Generate the signal `s = sin(2*pi*440*t)` and plot it.
 - c. Generate a second signal `s2 = sin(2*pi*5*t)` (a lower-frequency sine wave) and plot `s + s2` in the same figure, with `hold on` and two different colors. The two signals add sample by sample — try to explain why this works.
 - d. Repeat point (a) with the steps 0.1, 0.01 and 0.001, and observe how the *sampling* of the signal changes. How does the number of samples depend on the step?
 5. Logical indexing:
 - a. Create the vector `v = [1, 8, 3, 9, 5, 7]`.
 - b. Compute how many elements of `v` are larger than 5, using `sum(v > 5)` and a comment explaining the trick: the logical values `true/false` are counted as 1/0 by `sum()`.
 - c. Extract all the elements of `v` that are larger than 5, using logical indexing, and verify that their count matches the answer at point b.

4 Final questions

1. What does a semicolon at the end of a command do? When is it useful?
2. What is the difference between `a*b` and `a.*b`? Give an example of each.
3. What is the difference between a row vector and a column vector in Matlab, and how is each created?
4. How do you access the third element of a vector `v`? How do you access the last element of a vector of unknown length?
5. What does the command `whos` do? What information does it give?
6. When is `stem()` preferable to `plot()`?